CSC343H5 Lecture 8

Indexes and Query Optimization

Not Naaz Sibia (naaz.sibia@utoronto.ca)

Hello, I’m not Naaz!

But I am suppose to be here!

Naaz is away on conference, so I am covering today’s lecture.

I’m Amber, one of your TAs. I graduated from UTM in June, and am beginning my PhD at McMaster in September.

Admin

- Midterm in lecture next week (July 22nd)
  - Content is only stuff before the break!
- Office hours today (11-am-12pm) cancelled
- Naaz has online office hours on Friday from 12-3PM
  - EXTENDED HOURS - see Piazza @103
- Checkpoint 4 due July 26th

Please direct questions to Piazza/Naaz in OH.

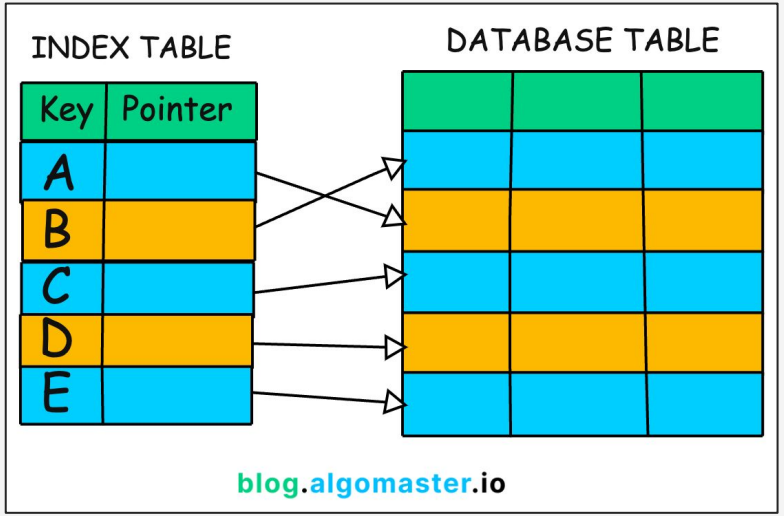
Recap Quiz

New Material Roadmap!

1. What are indexes?
2. Create an index in Postgres
3. Classify an indexes: keys and physical tuple ordering
4. Tree-based indexes
    a. ISAM
    b. B+ trees (in depth)
5. If we have time…
    a. Hash based indexes
    b. Impact of database tuning and index choice

Indexes

An index is a data structure that speeds access to tuples of a relation, given values of one or more attributes.

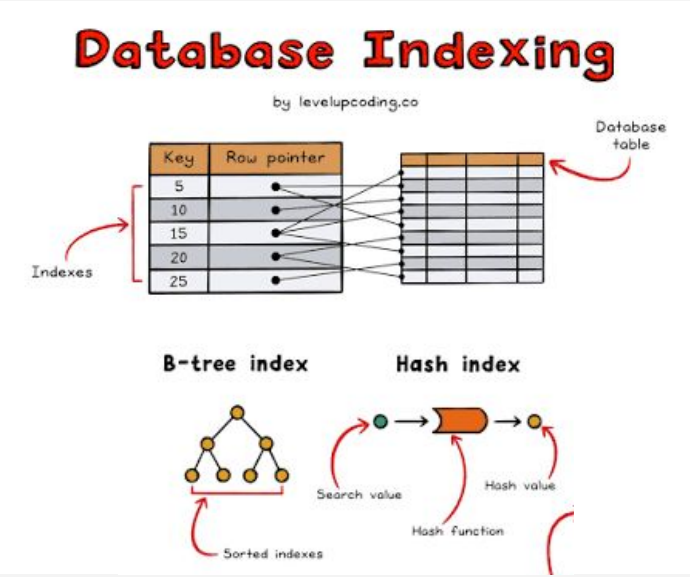


Why?

- Scanning all tuples is expensive
- This allows us to find pointers to what we want, faster!

Two Types of Indexes

Can be Hash Tables (like you've probably seen in prior courses) or (more commonly in DBMS) a balanced B-Tree



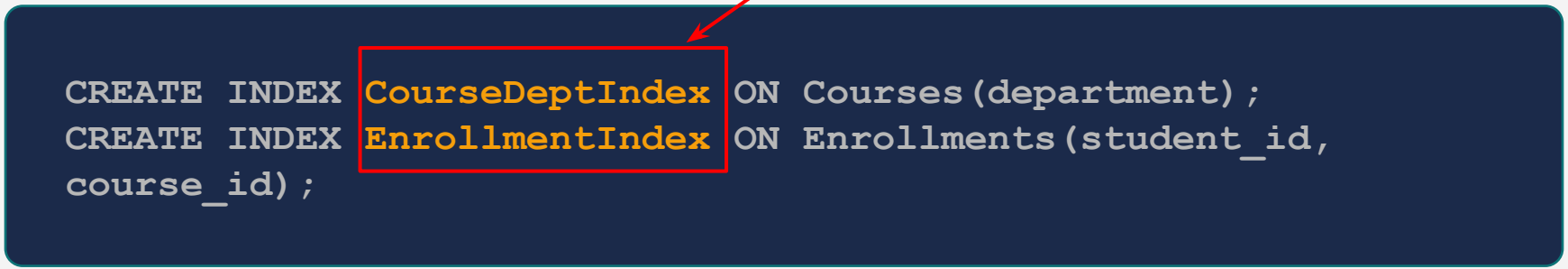
Declaring Indexes

Postgres Syntax:

```
CREATE INDEX CourseDeptIndex ON Courses(department);
CREATE INDEX EnrollmentIndex ON Enrollments(student_id,
course_id);
```

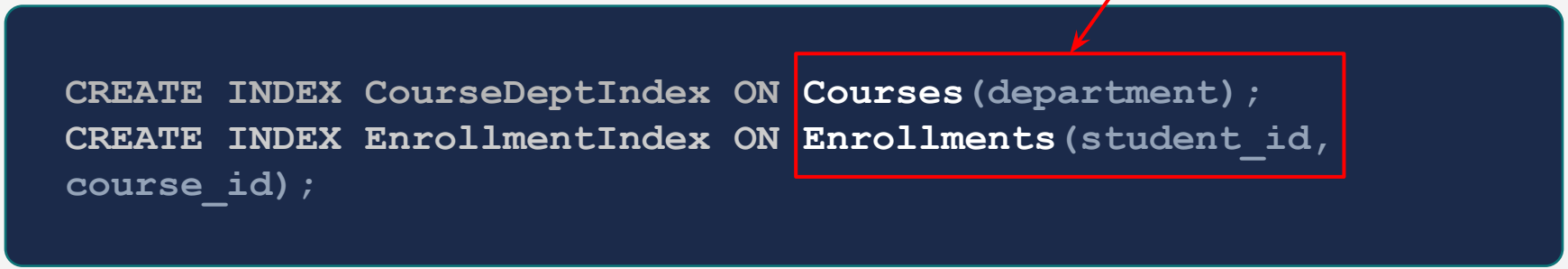
Declaring Indexes

Postgres Syntax:



Declaring Indexes

Postgres Syntax:



Using Indexes

Given a value v, the index takes us to only those tuples that have v in the attribute(s) of the index.

Example: find all CS grades from student S1234567

```
CREATE INDEX CourseDeptIndex ON Courses(department);
CREATE INDEX EnrollmentIndex ON
Enrollments(student_id, course_id);


SELECT grade FROM Courses, Enrollments
WHERE department = 'Computer Science'
  AND Courses.course_id =
Enrollments.course_id AND student_id =
'S1234567'
```

1. **Use CourseDeptIndex (on department)**
   Jump to all Courses rows where department = 'Computer Science'
2. **Use EnrollmentIndex (on student_id, course_id)**
   For each course found, jump to Enrollments rows where student_id = 'S1234567' and the course_id matches.
3. **Return grade**
   No full table scan needed!

Declaring Indexes

Postgres Syntax:

```
CREATE INDEX CourseDeptIndex ON Courses(department);
```

Optional Modifiers:

- **UNIQUE :** no duplicate values in the indexed column(s)
- **CLUSTERED:** sorts data rows to match index order (one per table)
- **USING HASH:** uses a hash structure instead of the default B+ tree

Syntax for Modifiers:

```
--Unique and clustering
CREATE [UNIQUE] [CLUSTERED] INDEX name ON Table (col1, ...) ;

--Another option to cluster:
CLUSTER Table USING IndexName;

--Using hash tables
CREATE INDEX IndexName ON Table USING hash (col1, ...);
```

Index Classification

Primary vs. Secondary

Primary Index

Search key contains the primary key of the relation.

Unique Index

Search key contains a superkey (guarantees uniqueness).

Secondary Index

Any other index; multiple secondary indexes can exist on one table.

Clustered vs. Unclustered

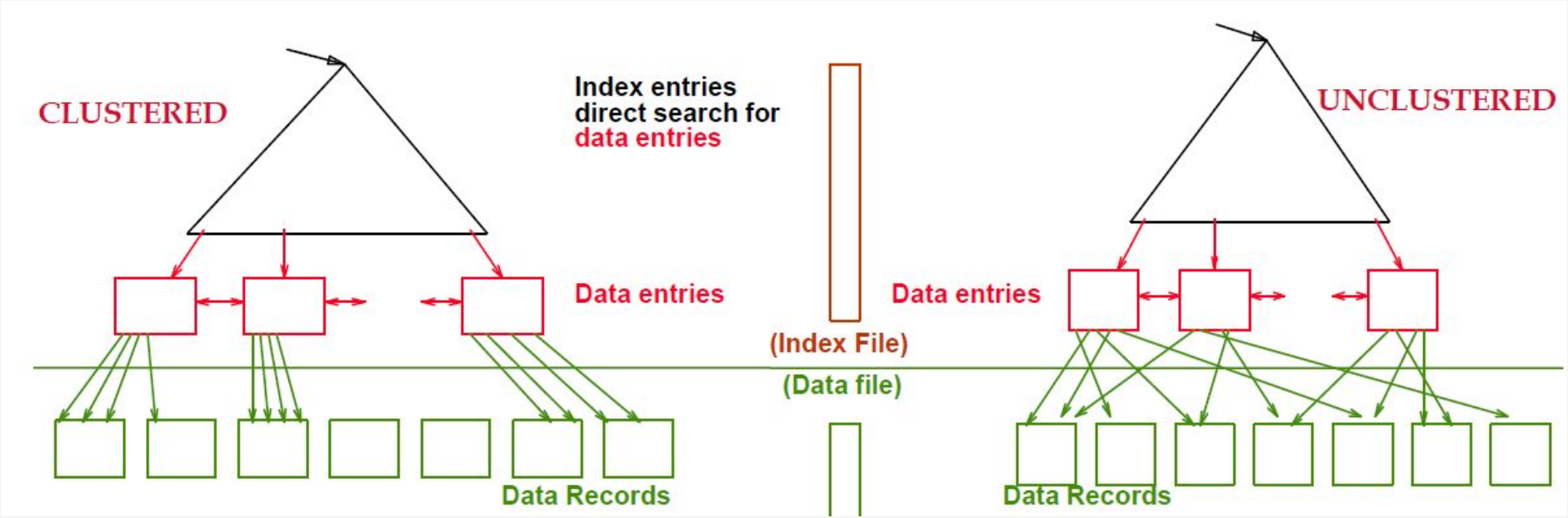
Clustered

Order of index data entries matches order of data records on disk. At most one per table.

Unclustered

Index order differs from data order. Multiple per table. Range scans are more expensive.

Index Classification



More Examples...

Given the below relation and index declaration, answer the following...

Is it a primary, unique, or secondary index?

Is it clustered or unclustered?

Hash table or B+ tree?

How is it ordered?

```
Student(student_id, name, DOB, currently_enrolled)

CREATE INDEX StudentInd ON Student(name, student_id);
```

More Examples...

Given the below relation and index declaration, answer the following…

Is it a primary, unique, or secondary index?
Unique (student_id is our primary key, so StudentInd is a superkey!)
Is it clustered or unclustered?
Unclustered, as it's not specified!
Hash table or B+ tree?
B+ tree, as that's the default!
How is it ordered?
By name, then if two students have the same name, by student_id!

```
Student(student_id, name, DOB, currently_enrolled)

CREATE INDEX StudentInd ON Student(name, student_id);
```

Handout time!

Complete box 1. If you’re confused, raise your hand!

Trees-Based Indexes

One of the most basic ideas is to create a file with one record per page, of the form <key on page, pointer to page> as index entries.

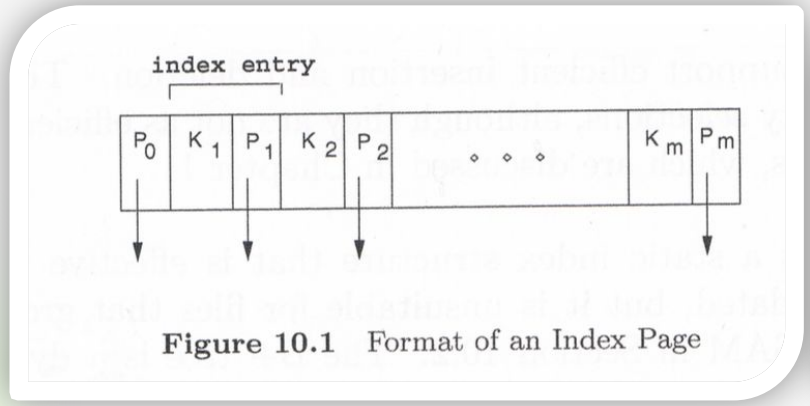


Figure 10.1 Format of an Index Page

The simple index file data structure is illustrated in Figure 10.2.

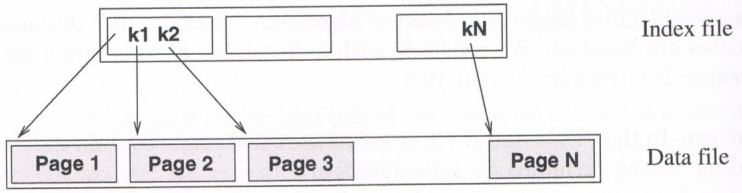


Figure 10.2 One-Level Index Structure

Indexed Sequential Access Method

ISAM is a data structure which retains its data entries in the leaf pages of the tree and additional *overflow* pages chained to some leaf page. This structure never changes except in the *overflow* pages.

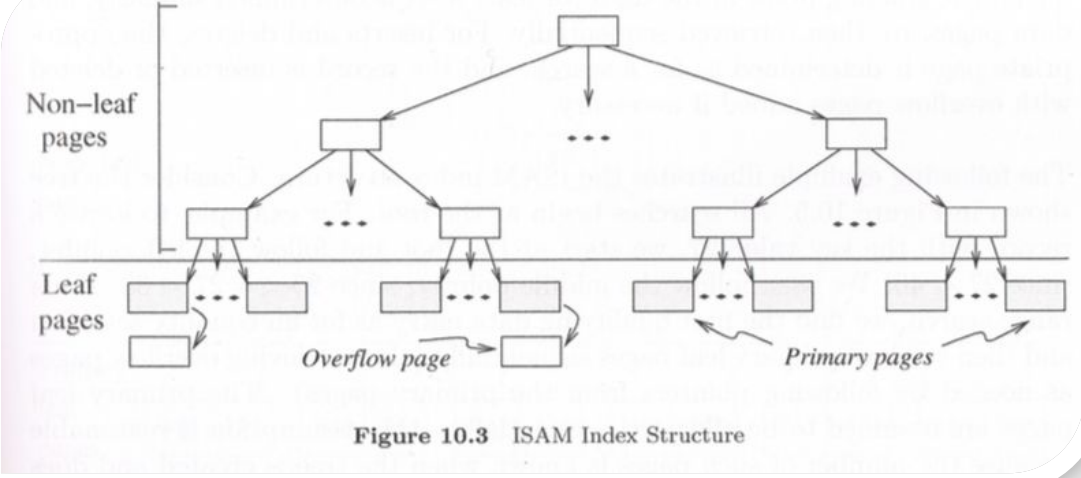
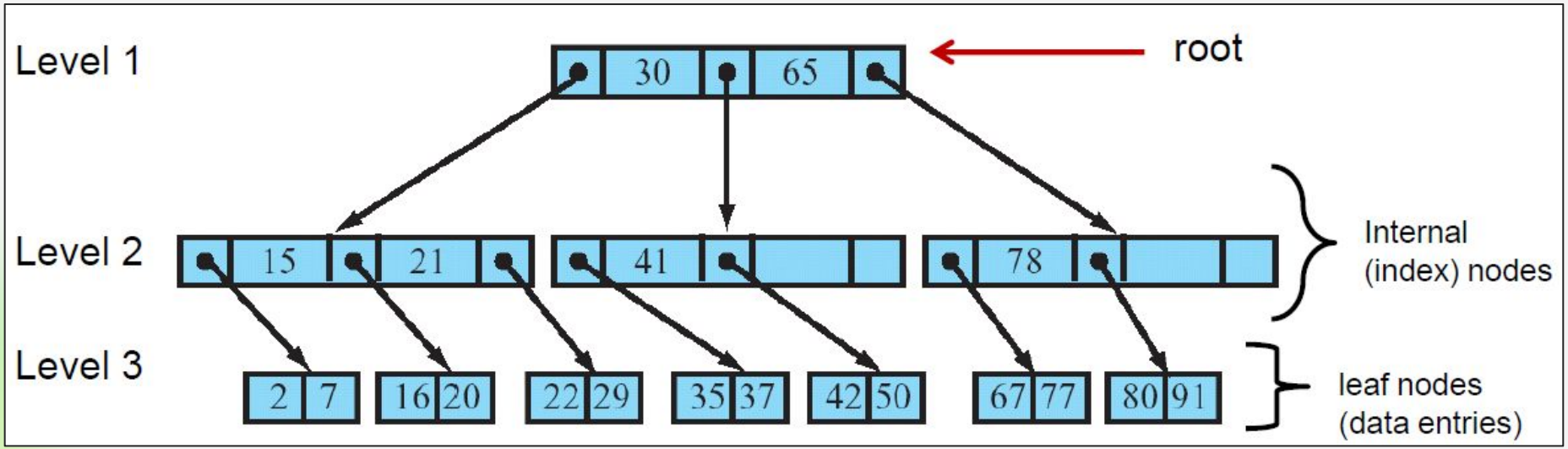


Figure 10.3 ISAM Index Structure

B+ Trees

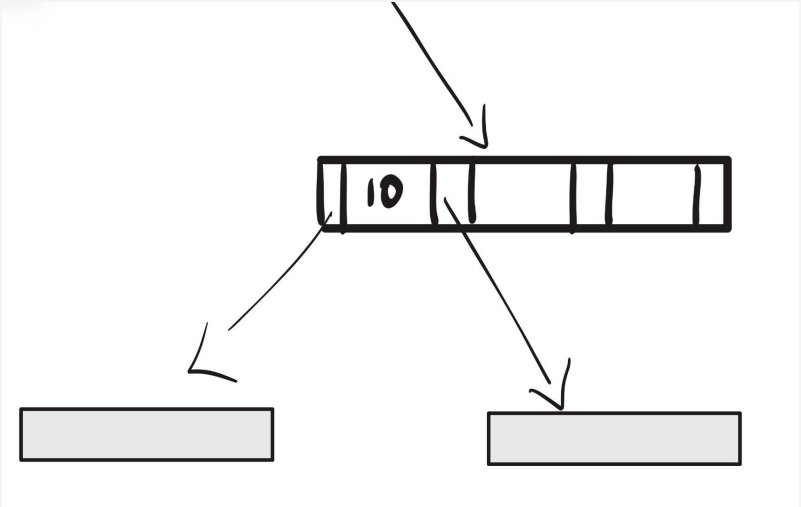
- The most common index type in databases today.
- As index files can be quite large, often stored on disks, partially loaded into memory as needed.



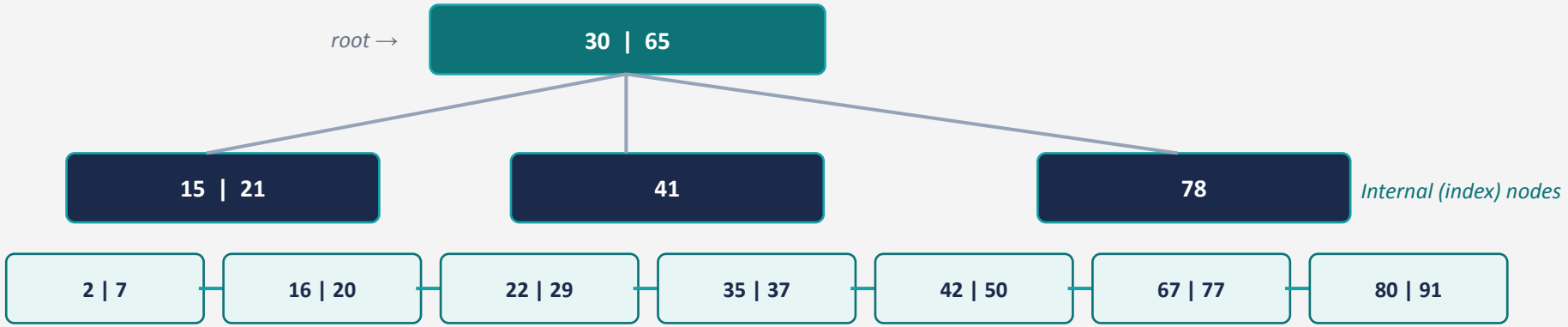
B+ Tree 50% rule

- Each node is at least 50% full.
- This applies to pointers, not keys.
- This does not apply to the root node either.

The image on the left does NOT violate the 50% rule, because 2/4 of the pointers are filled, even though only 1/3 keys are filled.



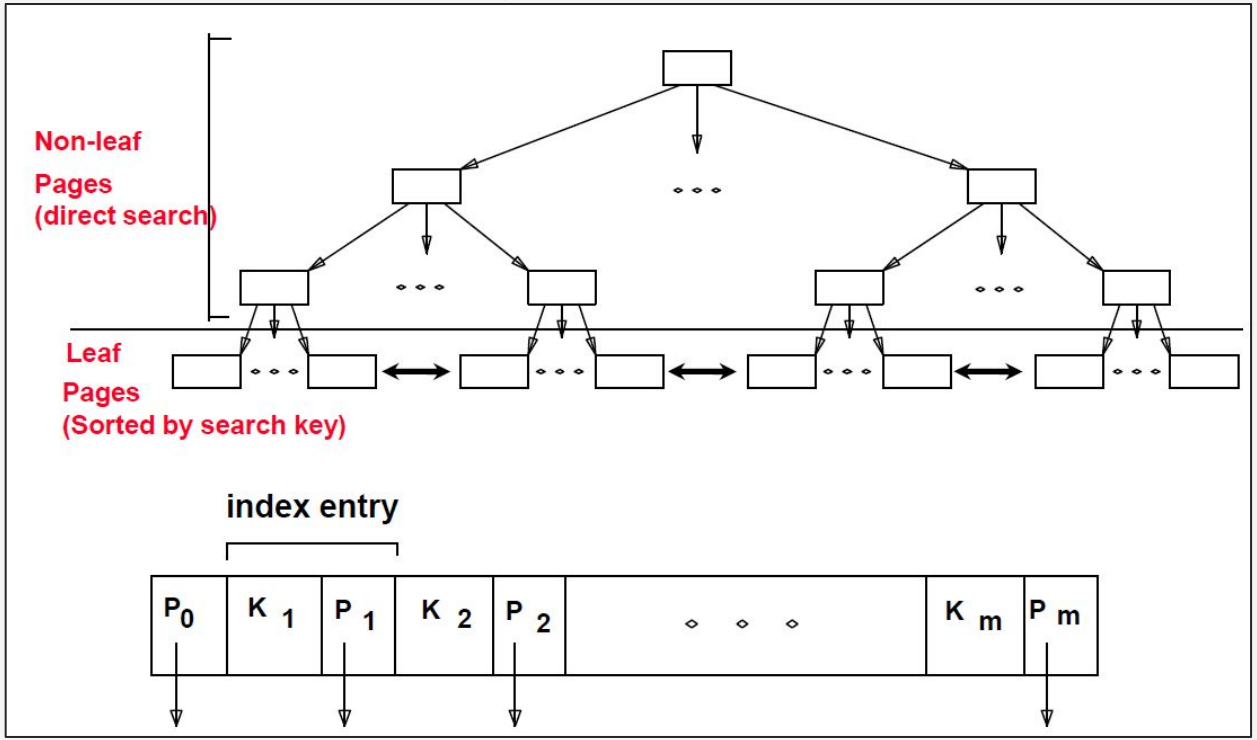
B+ Tree Structure



- **Non-leaf nodes:** contain only keys and pointers to child nodes
- **Leaf nodes:** contain the actual data entries (or pointers to them), sorted by search key, linked for range scans

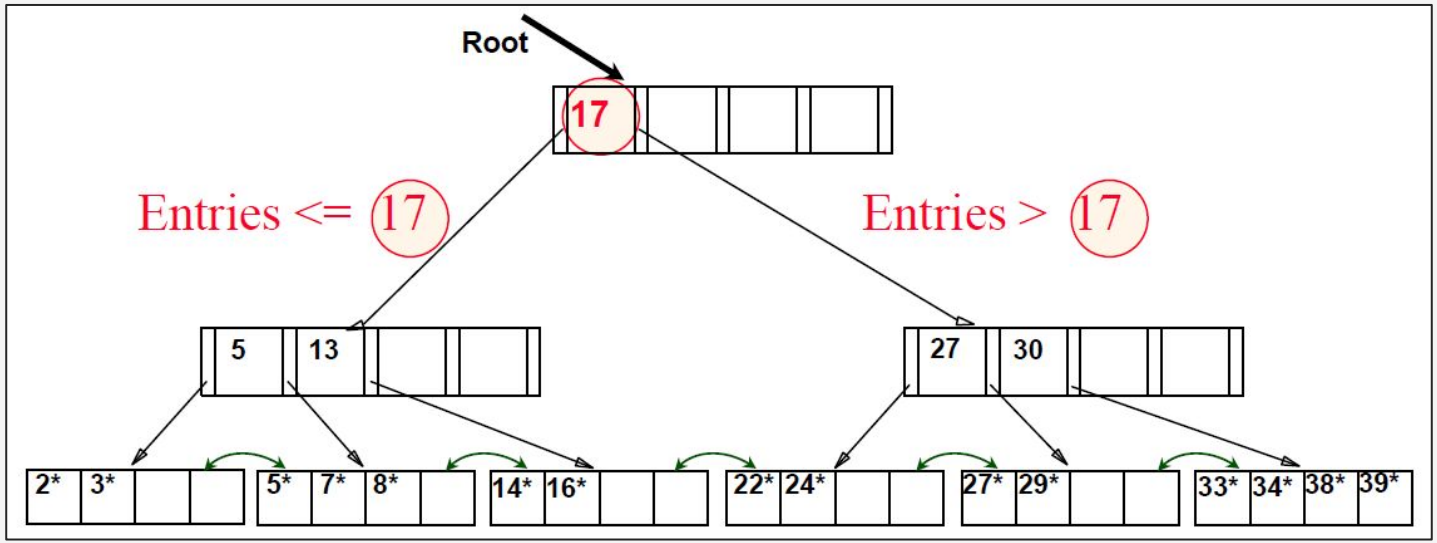
B+ Trees Searching

Supports equality and range-searches efficiently!



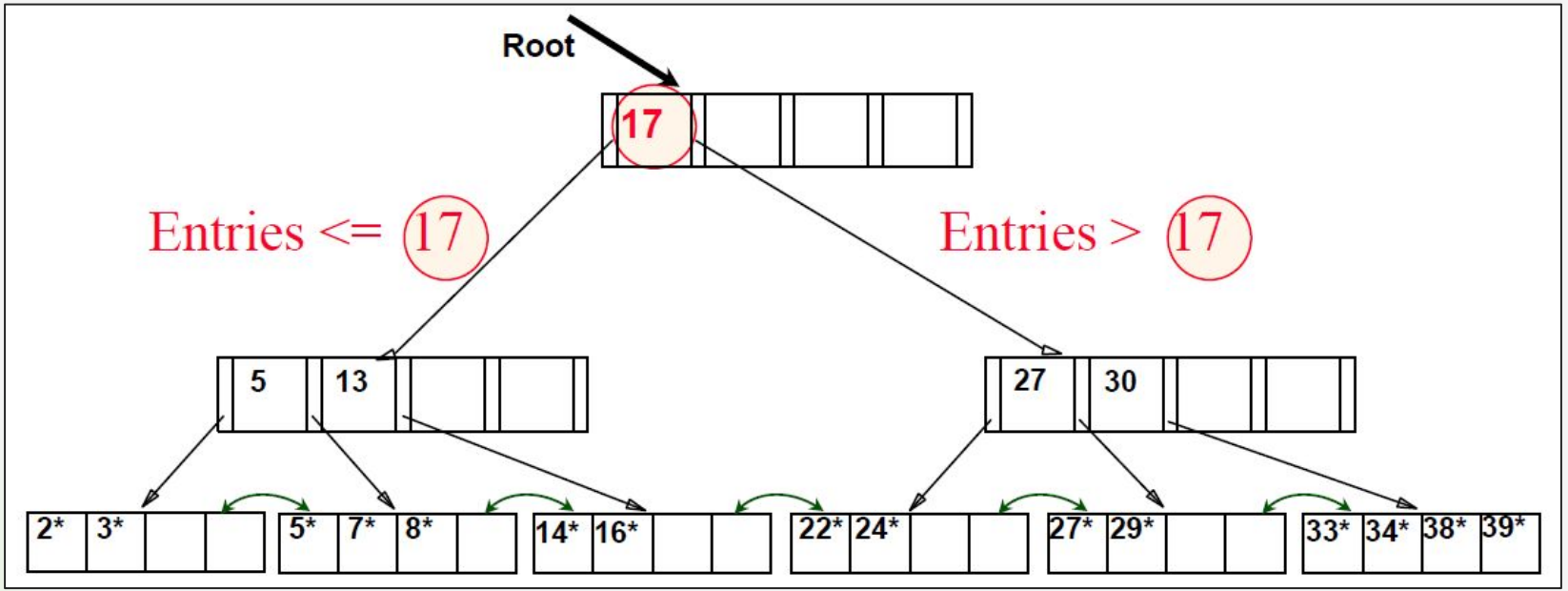
Examples: B+ Tree Search/Modify

1. Let's say that we are looking for 28*? 29*? All > 15* AND < 30*
2. **Insert/Delete:** Find a data entry in a leaf node and modify it.



Example: B+ Tree Search

Let's look for 28*? 29*? All > 15* AND < 30*



B+ Tree: Inserting a Data Entry

1 **Find the correct leaf L**

Traverse from root using key comparisons

2a **L has space - done!**

Insert the entry and you're finished

2b **L is full → split L**

Create L and a new node L_2; redistribute entries evenly; copy up the middle key

3 **Insert entry pointing to L_2 into parent**

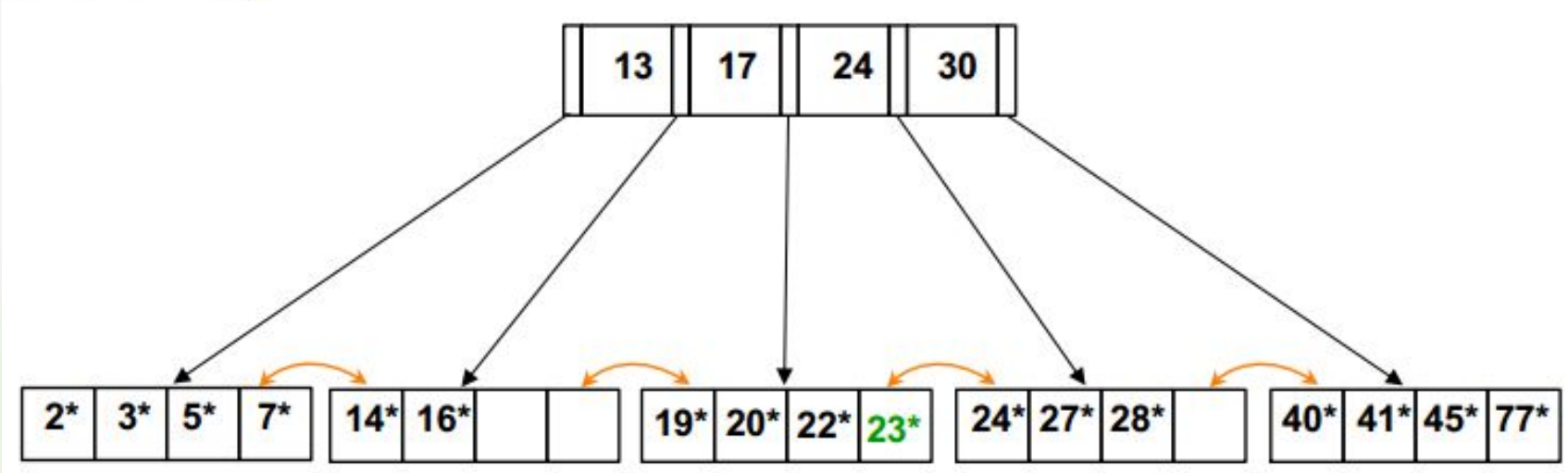
This can happen recursively all the way to the root

To split the index node, redistribute entries evenly, but push up the middle key. Splitting “grows” a tree; root split increases height.

B+ Tree Insertion Example

Example: to insert entry 23*

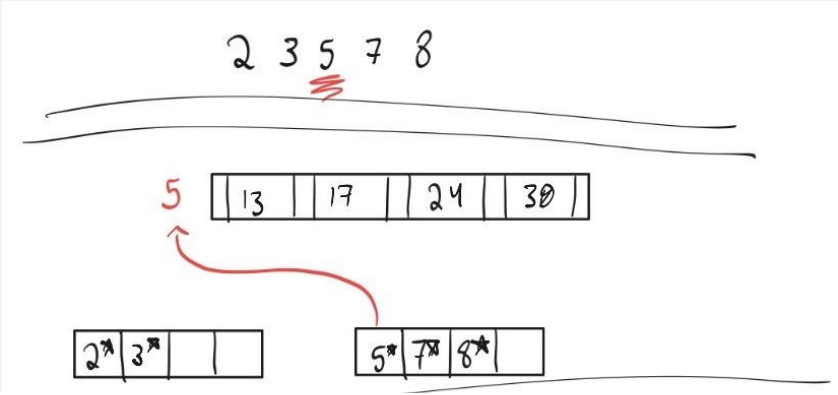
- Follow the third pointer, since 17 < 23 ≤ 24.
- Have enough space, done.

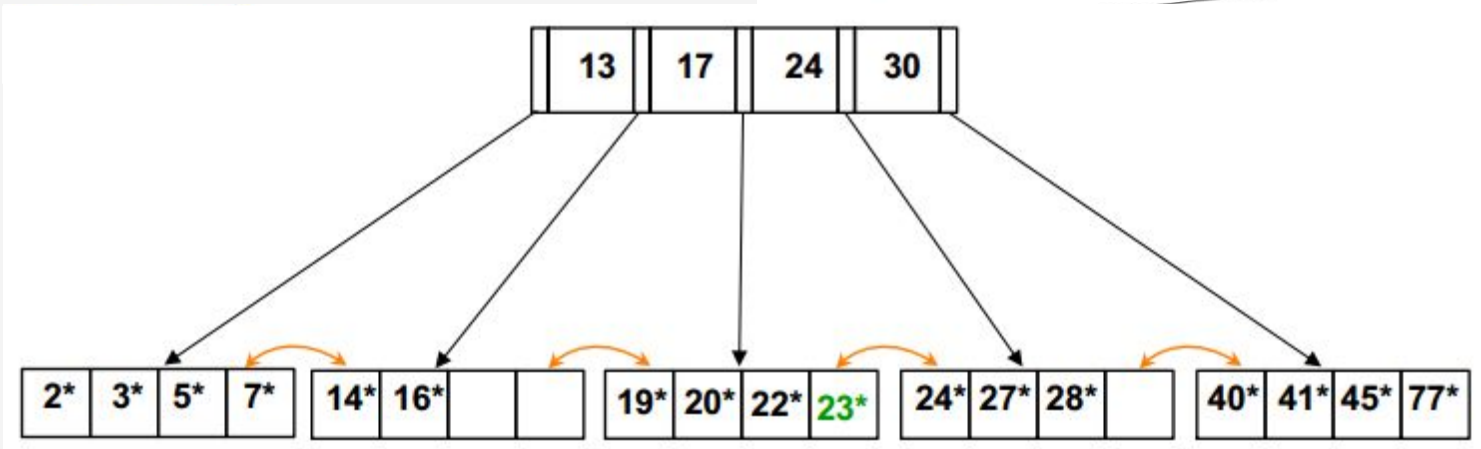


B+ Tree Insertion Example

Example: to insert entry 8*

- Follow the left-most pointer, since 8 < 13.
- Insertion causes overfill; split leaf node into two nodes and redistribute the data evenly.

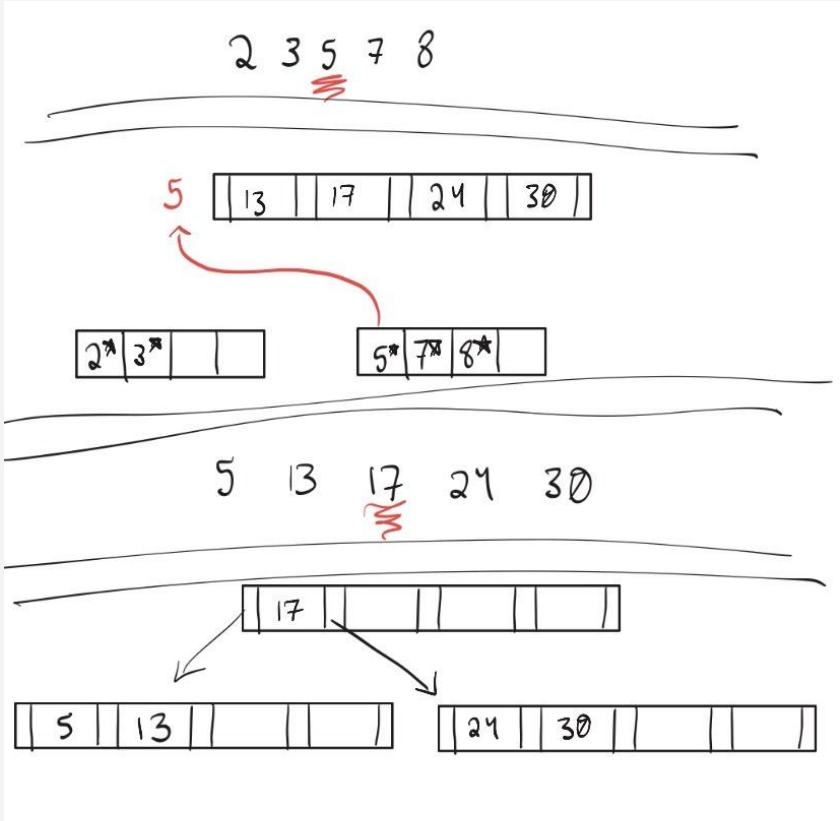




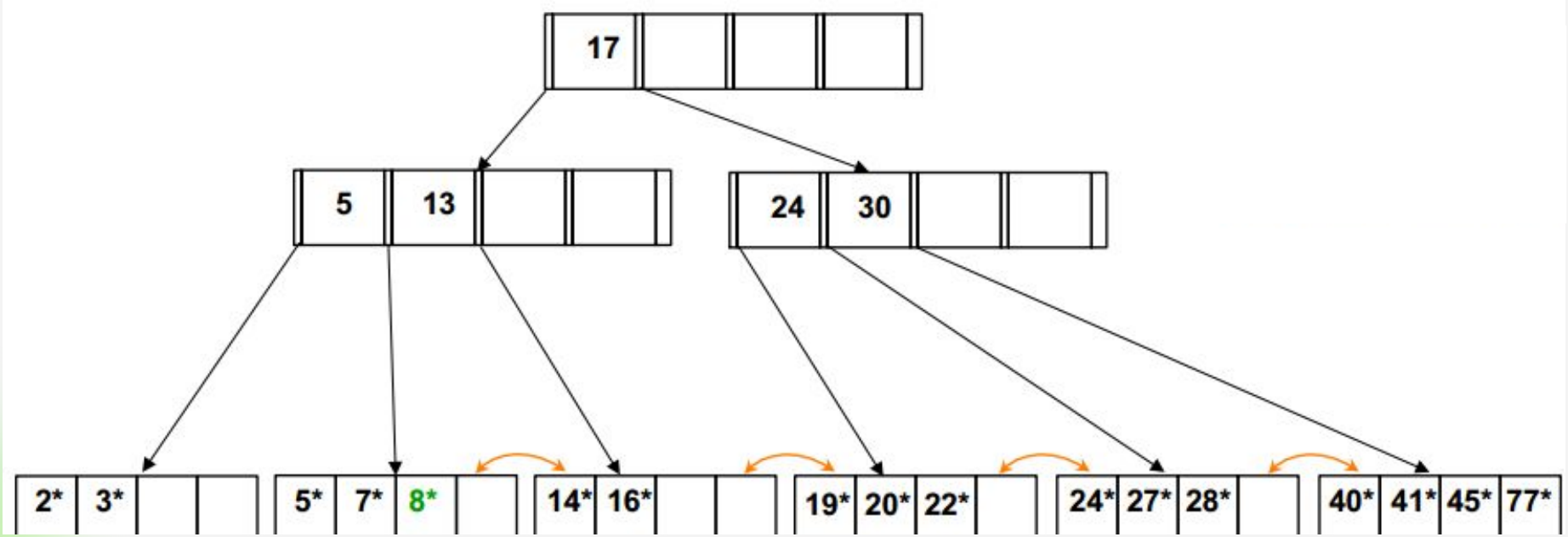
B+ Tree Insertion Example

Example: to insert entry 8*

- “5” is middle key, so it is copied up (keep the */leaf key, as this points to data).
- When inserting “5” into the parent node, it will cause overfill again. “17” is middle key and is pushed up (as 17 doesn’t point to data, remove it’s key from the child).



B+ Tree Insertion Example



B+ Tree: Deleting a Data Entry

1

Find the leaf L where the entry belongs

Traverse from root

2a

L is still ≥ half-full → done!

Remove entry, no restructuring needed

2b

L underflows → re-distribute

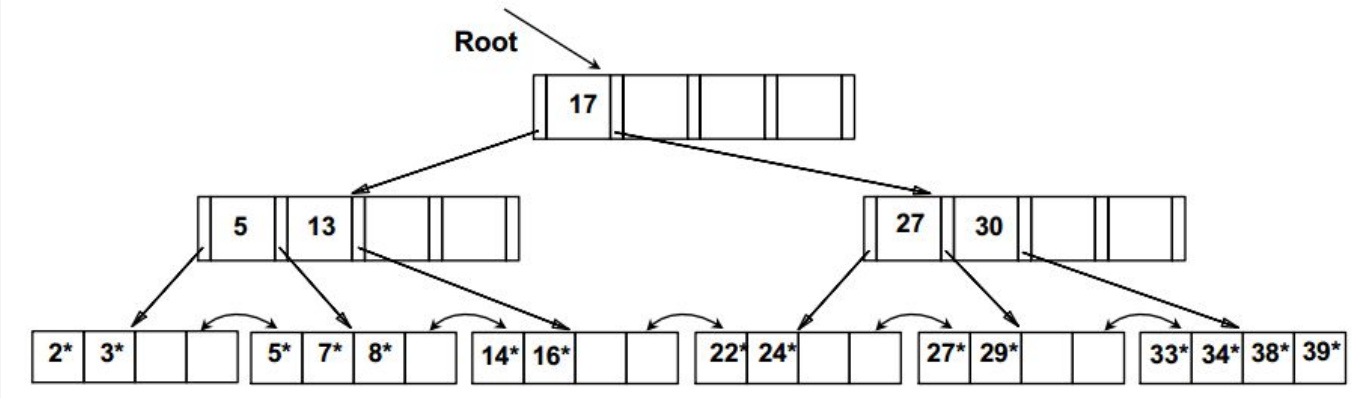
Borrow an entry from a sibling (adjacent node with same parent)

2c

Re-distribution fails → merge

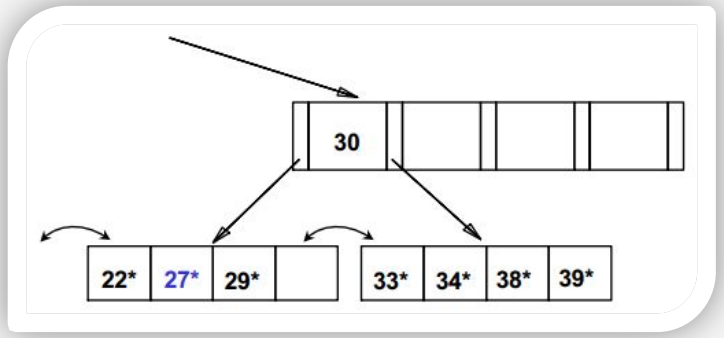
Merge L and sibling; delete the separating entry from parent (can propagate to root)

Example: Deleting from B+ Tree



Example: to delete entry 24*

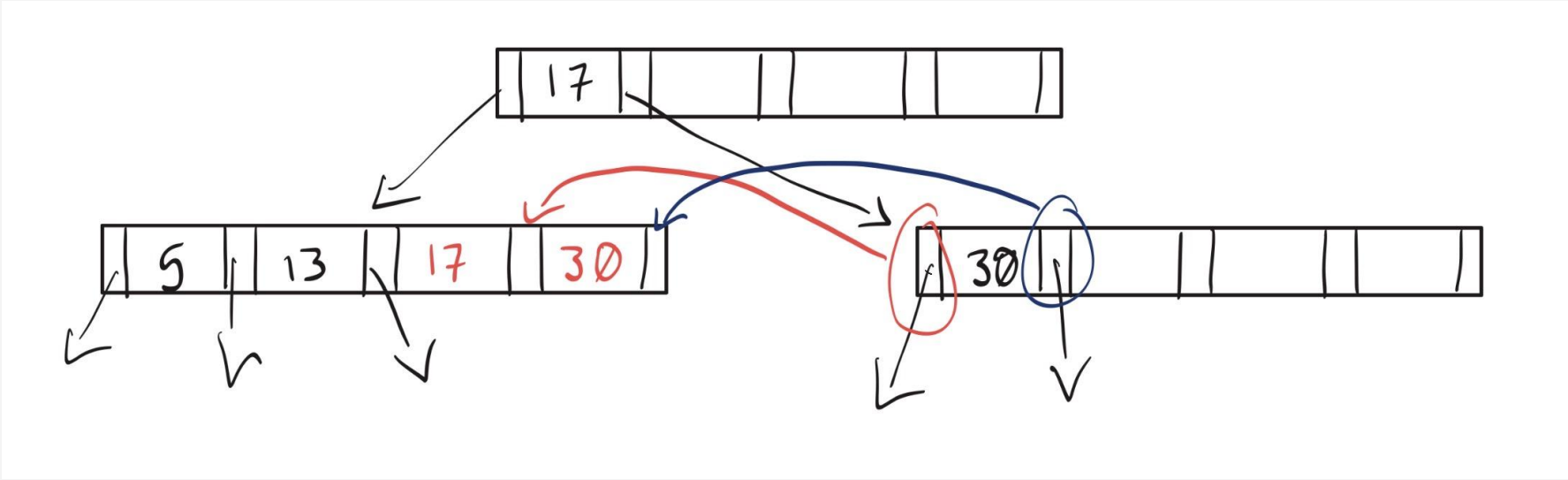
- After the deletion, the leaf contains only one entry 22*, and the sibling contains just two entries. Therefore, we can not redistribute entries.
- Merge leaf and sibling. ‘Toss’ the entry ‘27’ in the parent.
- Pull down if index entry



Example: Deleting from B+ Tree

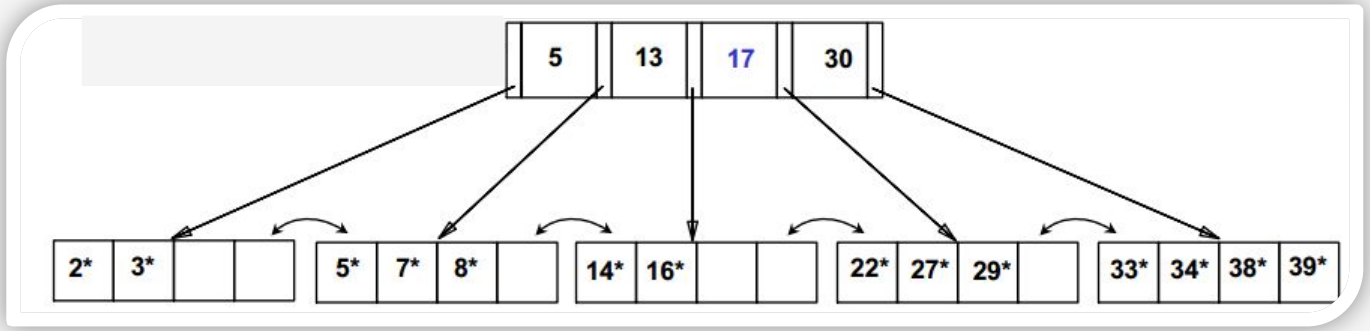
Example: to delete entry 24*

- After the deletion of 27, the left node contains only one entry, and the sibling contains just two entries. Therefore, we can not redistribute entries.
- Merge leaf and sibling. To do so, we need to “pull down” the 17



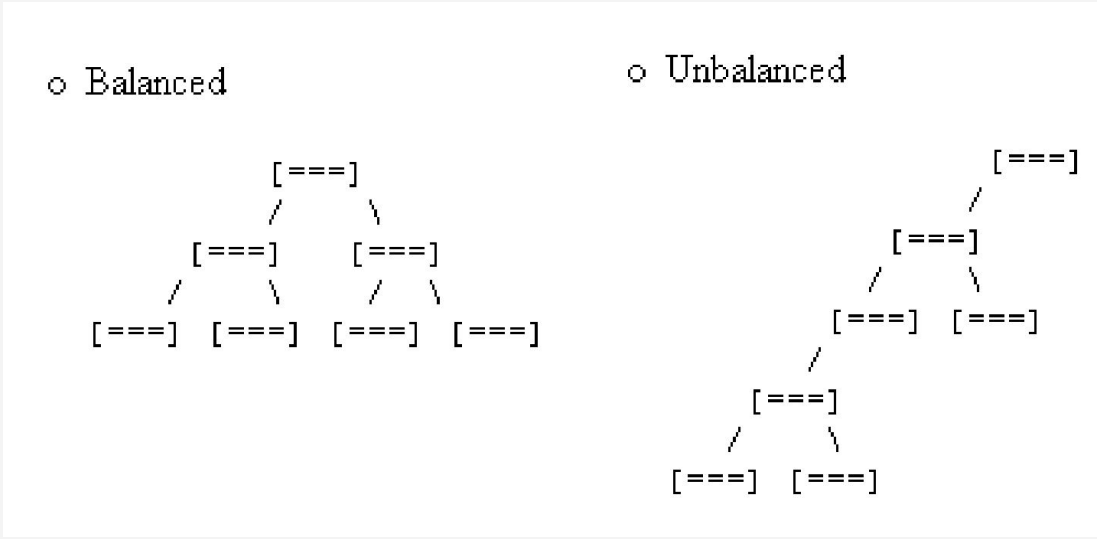
Example: Deleting from B+ Tree

Example: after deleting entry 24*



Balanced vs. Unbalanced Trees

- In a balanced tree, every path from the root to a leaf node is the same length.
- What is a B+ tree?



Handout time!

Box 2 time!

How are we feeling about B+ trees?

Our options:

1) More review of B+ trees
2) Move on to hash-based indexes

Hash-based Indexes

Fast and efficient for equality selections. The index is a collection of buckets.

How It Works

Hashing Function h

h(r) = bucket where record r belongs. h looks at the search key field of r.

Bucket = primary page + zero or more overflow pages. All contain data entries.

To search: apply h to identify the bucket, then search only that bucket.

Pros

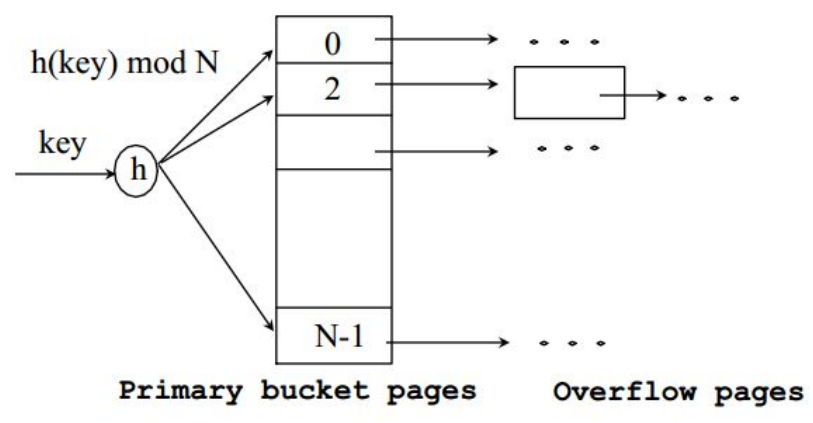
- Very fast for equality queries
- No sequential scan needed

Cons

- Does NOT support range queries
- Overflow pages degrade performance
- Choose h and bucket count carefully

Hash-based Indexes

Hash-based indexes are best for *equality selections*. They do not support efficient range searches.



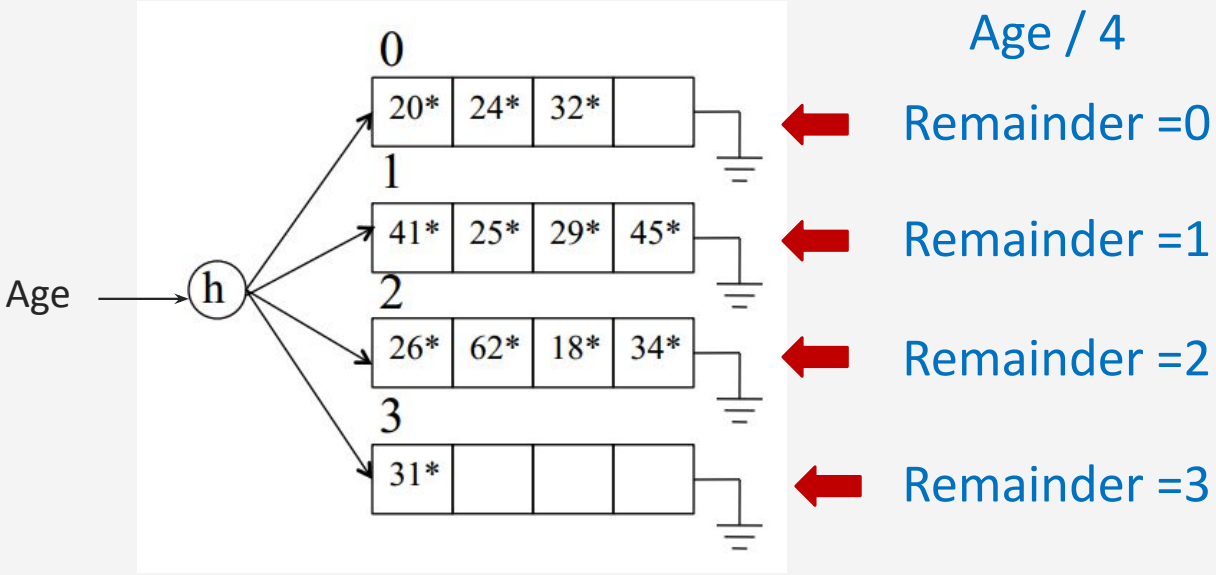
- a collection of buckets 0 through N-1, with one primary page per bucket initially
- bucket = **primary page**+ zero/more **overflow pages**
- buckets contains **data entries**

40

To search for a data entry, we apply a *hash function* ***h*** to identify the bucket to which it belongs and then search this bucket.

Example: Hash-based Indexes

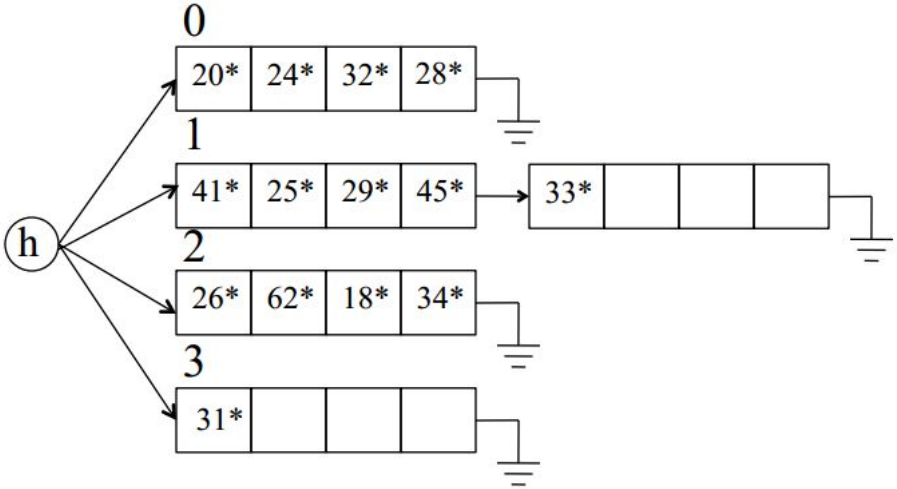
Initially constructed over the “Age” attribute, with 4 records/page and *h*(Age) = Age mod 4.



Example: Hash-based Indexes

To insert a data entry, we use the hash function to identify the correct bucket and then put the data entry there. If there is no space for this data entry, we allocate a new overflow page, put the data entry on this page, and add the page to the overflow chain of the bucket.

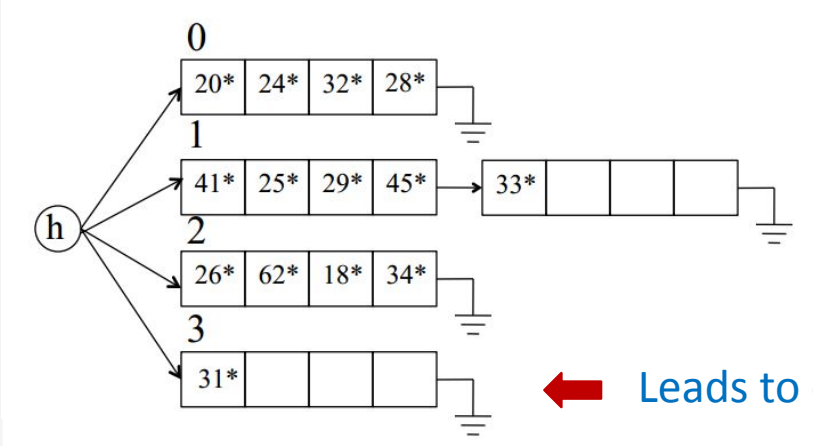
Example: adding 28, 33



Example: Hash-based Indexes

To delete a data entry, we use the hashing function to identify the correct bucket, locate the data entry by searching the bucket, and then remove it. If this data entry is the last in an overflow page, the overflow page is removed from the overflow chain of the bucket and added to a list of free pages.

Example : deleting 31



Leads to empty page

Handout time!

Finish off the worksheet (Box 3)

Database Tuning

A major problem in making a database run fast is deciding which indexes to create.

Benefit

An index speeds up queries that can use it. Particularly valuable on frequently queried columns.

Fallbacks

An index slows down all modifications on its relation (INSERT / UPDATE / DELETE) because the index must be updated too.

💡 Often the most useful index is on the relation's key, as it is queried frequently. Always weigh query speedup against write slowdown.

Workload Considerations

- Which relations and attributes are accessed most?
- Are queries equality-based (→ hash index) or range-based (→ B+ tree)?

Choice of Indexes

A tactic, often used, is to consider the most important queries in order. Consider the best plan using the current indexes and see if a better plan is possible with an additional index.

- This implies an understanding of how a DBMS evaluates queries and creates query evaluation plans.

- Before creating a new index, consider the impact on updates in the workload! There is often a trade-off; indexes can make queries go faster but make updates slower. Additionally, they require disk space!

Break Time

That’s a wrap!

Questions?